CS-E4960 - Software Testing and Quality Assurance D, Lecture, 3.9.2024-26.11.2024

This course space end date is set to 26.11.2024 **Search Courses: CS-E4960**

Unit Testing 2

To do: Make a submission **To do:** Receive a grade

Opened: Tuesday, 24 September 2024, 10:00 AM
Due: Tuesday, 1 October 2024, 10:00 AM

Overview

In the last assignment, you wrote unit tests for the number guessing game and answered questions on key testing concepts. Now, you will take a deeper dive into testing by focusing on statement coverage and modified condition/decision coverage (MC/DC) using the `coverage` library.

You will work on the same number guessing game application from the first assignment, but you will focus specifically on testing the scoreboard functionality implemented in `scoreboard.py` with the goal of achieving specific statement and MC/DC coverage.

The purpose of this assignment is to help you gain a more comprehensive understanding of coverage metrics and how they can be applied to unit testing, ensuring that all possible execution paths and statements in your code are covered by your tes ts.

You will start by answering conceptual questions related to coverage. Then, you will implement unit tests for the application using these concepts.

Note: For more details on the concepts covered in this assignment, check lecture materials.

Game Rules

- You start with 100 bet points.
- Choose a difficulty level:
  - Easy: 10 guesses
  - Hard: 5 guesses
- Place a bet with your bet points for each round.
- Guess the number between 1 and 100:
  - If you guess correctly, you can choose to double your prize or take your winnings.
  - If you choose to double, you have a 50% chance to either double your prize or lose the amount you have won, meaning your total prize remains unchanged.
- If you run out of guesses, you can try to save your bet:
  - You have a 50% chance to keep your bet points or lose double the amount you placed for that round, meaning you lose twice the bet points if you lose, on top of what you've already lost.
- After each round, you can decide to continue playing or finish the game and save your score to the scoreboard.
- The game ends when you choose to finish or you have no bet points left.
- The high scores are saved and displayed on the scoreboard. Try to get your name in the top 10!

How to Play the Game

Web Version:

- Run the Web Application:
  - Open a terminal and navigate to the project directory.
  - Run the following command: `python app.py`
  - Open your web browser and go to `http://127.0.0.1:5000/`.

CLI Version:

- Run the CLI Application:
  - Open a terminal and navigate to the project directory.
  - Run the following command: `python game.py`
  - Follow the prompts:
    - Enter your name.
    - Choose a difficulty level.
    - Place bets and guess numbers as prompted in the terminal.

How to Install Dependencies

- Ensure you have Python installed on your system.
- Open a terminal and navigate to the project directory.
- Run the following command to install the required dependencies:

```
pip install -r requirements.txt
```

How to Write and Test Unit Tests

There is a test file named `test_scoreboard.py`.

Implement Test Cases:

- Write tests for the functions in `scoreboard.py` focusing on statement coverage and MC/DC.
- For more details on the concepts covered in this assignment, refer to the lecture materials.

Conceptual Questions

Conceptual questions are provided in the `conceptual_answers.txt` file. You should write your answers into this file and include it in your submission with your tests.

Unit Testing Instructions

You will write tests for the following functions in `scoreboard.py`:

- **load_scores()**: Focus on achieving 100% statement coverage. While the goal is to aim for 100% statement coverage, there will be no penalty for submissions that fall slightly below this target (95%).
- **add_score()**: Focus on achieving MC/DC.
- **save_scores()**, **display_scores()**, and **get_scores()**: Write comprehensive tests for these functions.

Use the `test_scoreboard.py` file to implement your unit tests.

Coverage.py

When you implement your tests, use the `coverage.py` tool to measure the statement coverage and generate an HTML report.

For more information, check https://coverage.readthedocs.io/.

You can check the coverage of individual functions in the HTML report by navigating to the 'Functions' tab. Also, sure to explore the documentation page and find the command to check statement coverage using the coverage library.

File Structures of the Assignment:

- **NUMBER-GUESSING-GAME**
  - **templates**
    - bet.html
    - correct.html
    - difficulty.html
    - end.html
    - guess.html
    - index.html
    - out_of_guesses.html
  - **tests**
    - test_scoreboard.py
  - app.py
  - game.py
  - routes.py
  - scoreboard.py
  - conceptual_answers.txt
  - scoreboard.txt
  - requirements.txt

Submission: File Submission, i.e., upload the whole project folder as a zip file. Double-check that your zip file includes updated test files, html report for coverage and the `conceptual_answers.txt` file.

Grading:

- 4-3 points: The assignment is clear and insightful, with correct implementation of test cases and well-explained theory. Test cases are complete, and edge cases are covered.
- 3-2 points: The assignment is mostly clear but may lack some critical thinking. Test cases are implemented, but some important edge cases are missed.
- 1-0 points: The assignment lacks clarity or coverage of critical concepts. Test cases are missing or fail to cover key functions or edge cases.

Use of AI and Collaboration with Other Students: Using AI tools to produce an answer for this assignment is not permitted. All parts of the submission must be produced by yourself (apart from files provided in the assignment). Collaboration with other students is not permitted. This is an individual assignment that you must author independently.

UnitTesting2.zip 10 September 2024, 9:02 AM

Submission status

Attempt number	This is attempt 1.
Submission status	No submissions have been made yet
Grading status	Not marked
Time remaining	Assignment is overdue by: 162 days 4 hours
Last modified	-

Previous activity
← Unit Testing 1

Next activity
Unit Testing 3 →